

Cheat Sheet — Programmazione ed Algoritmica

4ª Prova in Itinere · Diego Stefanini

Programmazione Dinamica

Quando si applica

Due condizioni devono valere contemporaneamente:

- **Sovrapposizione dei sotto-problemi:** la ricorsione naturale ricalcola più volte gli stessi sotto-problemi.
- **Sottostruttura ottima:** la soluzione ottima del problema si costruisce da soluzioni ottime di sotto-problemi.

PD vs Divide et Impera: D&I suddivide in sotto-problemi **indipendenti**; PD sfrutta invece la **sovrapposizione** memorizzando i risultati.

PD vs Greedy: Greedy fa scelte localmente ottime senza tornare indietro; PD esplora tutte le scelte tramite la ricorrenza e sceglie la migliore.

Due approcci equivalenti

- **Top-down (memoization):** ricorsione + tabella di cache. Scrivi direttamente la ricorrenza, ricordando ciò che hai già calcolato.
- **Bottom-up (tabulation):** ciclo iterativo che riempie la tabella nell'ordine giusto (dai casi base verso il problema completo). Di solito più efficiente in pratica.

Schema generale di risoluzione

1. **Definisci DP[...]:** cosa rappresenta ciascuna cella (soluzione ottima/booleana di quale sotto-problema).
2. **Ricorrenza:** esprimi DP[...] in funzione di celle "più piccole".
3. **Casi base:** celle risolvibili direttamente senza ricorrenza.
4. **Ordine di riempimento:** tale che quando calcoli una cella i suoi dipendenti sono già pronti.
5. **Estrai la risposta** dalla cella finale.
6. **Ricostruzione (opzionale):** tabella ausiliaria $\text{pred}[\dots]$ che registra la scelta vincente in ogni cella; risali dalla cella finale seguendo la catena.

Taglio della Corda (Rod Cutting) — $\Theta(n^2)$

Asta di lunghezza n , prezzi $p[1..n]$ per pezzi interi. Massimizzare il ricavo.

Ricorrenza: $r[0] = 0$, $r[j] = \max_{1 \leq i \leq j} (p[i] + r[j - i])$.

```
extendedBottomUpCutRod(p, n):
    r[0] = 0
    for j = 1 to n:
        q = -infinity
        for i = 1 to j:
            if q < p[i] + r[j-i]:
                q = p[i] + r[j-i]; s[j] = i
        r[j] = q
    return r, s
```

Ricostruzione: stampa $s[n]$, poi passa a $n - s[n]$, ripeti finché resto = 0.

```
printCutRod(p, n):
    (r, s) = extendedBottomUpCutRod(p, n)
    while n > 0:
        print s[n]
        n = n - s[n]
```

Complessità: tempo $\Theta(n^2)$, spazio $\Theta(n)$.

Sottosequenza Comune Più Lunga (LCS) — $\Theta(mn)$

Date $X[1..m]$ e $Y[1..n]$, trovare la più lunga sequenza Z che sia sottosequenza di entrambe.

Ricorrenza: $c[i, j]$ = lunghezza LCS di $X[1..i]$ e $Y[1..j]$.

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ c[i - 1, j - 1] + 1 & \text{se } X[i] = Y[j] \\ \max(c[i - 1, j], c[i, j - 1]) & \text{altrimenti} \end{cases}$$

```
lcsLength(X, Y):
    m = X.length; n = Y.length
    for i = 0 to m: c[i][0] = 0
    for j = 0 to n: c[0][j] = 0
    for i = 1 to m:
        for j = 1 to n:
            if X[i] == Y[j]:
                c[i][j] = c[i-1][j-1] + 1
                b[i][j] = "DIAG"
            elif c[i-1][j] >= c[i][j-1]:
                c[i][j] = c[i-1][j]; b[i][j] = "UP"
            else:
                c[i][j] = c[i][j-1]; b[i][j] = "LEFT"
    return c, b
```

Ricostruzione (percorso a ritroso in b da (m, n)): $\text{printLCS}(b, X, i, j)$:

```
if i == 0 || j == 0: return
if b[i][j] == "DIAG":
    printLCS(b, X, i-1, j-1); print X[i]
elif b[i][j] == "UP":
    printLCS(b, X, i, j-1)
else:
    printLCS(b, X, i, j-1)
```

Complessità: tempo $\Theta(mn)$, spazio $\Theta(mn)$ (riducibile a $\Theta(\min(m, n))$ se non serve ricostruire).

Cammini su Griglia — Conteggio

Scacchiera $n \times n$, mosse giù o destra, contare i cammini da $(0, 0)$ a $(n - 1, n - 1)$.

Ricorrenza: $\text{DP}[i][j] = \text{DP}[i - 1][j] + \text{DP}[i][j - 1]$. **Casi base:** $\text{DP}[0][j] = 1$, $\text{DP}[i][0] = 1$ (un unico cammino sulla frontiera).

```
countPaths(n):
    for j = 0 to n-1: DP[0][j] = 1
    for i = 0 to n-1: DP[i][0] = 1
    for i = 1 to n-1:
        for j = 1 to n-1:
```

```
        DP[i][j] = DP[i-1][j] + DP[i][j-1]
    return DP[n-1][n-1]
```

Complessità: tempo e spazio $\Theta(n^2)$. (Formula chiusa: $\binom{2n-2}{n-1}$.)

Cammini su Griglia — Valore Massimo

Stessa griglia, ogni casella ha valore $c[i][j]$. Trovare il cammino con somma massima.

Ricorrenza: $\text{DP}[i][j] = c[i][j] + \max(\text{DP}[i - 1][j], \text{DP}[i][j - 1])$. **Casi base:** $\text{DP}[0][0] = c[0][0]$; prima riga/colonna = somme cumulative.

```
maxPath(C, n):
    DP[0][0] = C[0][0]
    for j = 1 to n-1: DP[0][j] = DP[0][j-1] + C[0][j]
    for i = 1 to n-1: DP[i][0] = DP[i-1][0] + C[i][0]
    for i = 1 to n-1:
        for j = 1 to n-1:
            DP[i][j] = C[i][j] + max(DP[i-1][j], DP[i][j-1])
    return DP[n-1][n-1]
```

Complessità: tempo e spazio $\Theta(n^2)$.

Longest Increasing Subsequence (LIS) — $O(n^2)$

Array $L[0..n - 1]$ di interi distinti. Trovare la più lunga sottosequenza strettamente crescente.

Ricorrenza (definizione "termina in i "): $\text{LIS}[i] = 1 + \max\{\text{LIS}[j] : j < i, L[j] < L[i]\}$, oppure 1 se nessun j soddisfa la condizione.

```
lisLength(L, n):
    for i = 0 to n-1:
        LIS[i] = 1; pred[i] = -1
        for j = 0 to i-1:
            if L[j] < L[i] && LIS[j] + 1 > LIS[i]:
                LIS[i] = LIS[j] + 1; pred[i] = j
    return max(LIS[0..n-1])
```

Ricostruzione: trova i^* che massimizza $\text{LIS}[i]$; segui $\text{pred}[i^*]$ a ritroso. Inverti la sequenza.

Complessità: tempo $\Theta(n^2)$, spazio $\Theta(n)$. (Versione $O(n \log n)$ esiste ma non è PD.)

Partition (Subset Sum) — $O(ns)$

Dato $A = \{a_1, \dots, a_n\}$ di interi positivi con somma $2s$, esiste $A' \subseteq A$ con somma s ?

Ricorrenza: $\text{DP}[i][j]$ = vero sse esiste sottoinsieme dei primi i elementi con somma j .

$$\text{DP}[i][j] = \begin{cases} \text{vero} & \text{se } j = 0 \\ \text{falso} & \text{se } i = 0 \text{ e } j > 0 \\ \text{DP}[i - 1][j] & \text{se } a_i > j \\ \text{DP}[i - 1][j] \vee \text{DP}[i - 1][j - a_i] & \text{altrimenti} \end{cases}$$

```
partition(A, n):
    totale = sum(A)
    if totale is odd: return false
    s = totale / 2
    for i = 0 to n: DP[i][0] = true
    for j = 1 to s: DP[0][j] = false
    for i = 1 to n:
        for j = 1 to s:
            if A[i] > j: DP[i][j] = DP[i-1][j]
            else: DP[i][j] = DP[i-1][j] OR DP[i-1][j - A[i]]
    return DP[n][s]
```

Complessità: tempo e spazio $O(ns)$ — **pseudo-polinomiale:** dipende dal valore di s , non dal numero di bit necessari a rappresentarlo. Partizione è NP-completo: se s cresce esponenzialmente nei bit dell'input, l'algoritmo non è trattabile.

Checklist d'esame (PD)

1. Enuncia DP[...] a parole (non solo in formule).
2. Scrivi la ricorrenza con tutti i casi (inclusi quelli di frontiera).
3. Specifica esplicitamente i casi base.
4. Giustifica l'ordine dei cicli.
5. Mostra il contenuto della tabella su un esempio piccolo.
6. Analizza tempo e spazio.
7. Se il problema chiede la soluzione ottima e non solo il valore, mostra la tabella pred e il procedimento di ricostruzione.